Transcriptomics Data Analysis - Part 2

Davide Bressan

2026-04-07

This is the second part of the webinar on "Transcriptomics Data Analysis" organised for the 2nd edition of the international BioHackathon of the Bioinformatics Italian Society (BITS) - 2026.

Introduction

In this second part of the tutorial we will perform some basic exploratory data analysis and learn how to normalize the **count** matrix.

- The count matrix is a table where rows represent genes and columns represent samples.
- Each entry in the matrix contains the number of reads that have been mapped to a particular gene in a specific sample.
- This raw count data is the starting point for many downstream analyses, including quality controls (PCA, clustering)

The bulk RNA-seq data are from an experiment investigating **pasilla** (or ps), a key regulator of alternative splicing that binds mRNA within the spliceosome (Brooks et al. 2011 - PMID: 20921232). The study utilized siRNA knockdown to observe how the loss of pasilla function affects the transcriptome, comparing three biological knockdown replicates against four untreated controls (NCBI GEO: GSM461176 to GSM461181).

Find here the R package pasilla.

1. Load the packages and the data

1.1 Install the required R packages

We start by installing the required R packages for this tutorial. If you already have these packages installed, you can skip this step.

```
if (!require("BiocManager", quietly = TRUE))
    install.packages("BiocManager")
BiocManager::install("ComplexHeatmap")
BiocManager::install("DESeq2")
install.packages("ggplot2")
install.packages("RColorBrewer")
install.packages("ggrepel")
```

Once the required R packages are installed, we can load them into our R session.

1.2 Load the required R packages

```
library(ComplexHeatmap)
library(DESeq2)
library(ggplot2)
library(RColorBrewer)
library(ggrepel)
```

Once the required R packages are loaded, we can load the count matrix and the sample information from the pasilla dataset.

1.3 Load the count matrix and the sample information

Load the raw count matrix from the pasilla dataset. The `system.file()` function is used to locate the files within the DESeq2 package folder.

```
#get the path to the raw count matrix
pasillaRawCounts_path <- system.file("extdata",
                      "pasilla_gene_counts.tsv.gz",
                      package="DESeq2", mustWork=TRUE)

#now we can read the count matrix from the file
pasillaRawCounts <- read.table(pasillaRawCounts_path, sep = "\t", header = TRUE, row.names = 1)

head(pasillaRawCounts,4)
```

```
##             untreated1 untreated2 untreated3 untreated4 treated1 treated2
## FBgn0000003          0          0          0          0        0        0
## FBgn0000008         92        161         76         70      140       88
## FBgn0000014          5          1          0          0        4        0
## FBgn0000015          0          2          1          2        1        0
##             treated3
## FBgn0000003        1
## FBgn0000008       70
## FBgn0000014        0
## FBgn0000015        0
```

```
dim(pasillaRawCounts)
```

```
## [1] 14599     7
```

Now the sample annotation:

```
pasAnno_path <- system.file("extdata", # nolint: object_name_linter.
                      "pasilla_sample_annotation.csv",
                      package="DESeq2", mustWork=TRUE)

pasAnno <- read.csv(pasAnno_path, row.names = 1)
pasAnno
```

```
##              condition        type number.of.lanes total.number.of.reads
## treated1fb     treated single-read               5              35158667
## treated2fb     treated  paired-end               2         12242535 (x2)
## treated3fb     treated  paired-end               2         12443664 (x2)
## untreated1fb untreated single-read               2              17812866
## untreated2fb untreated single-read               6              34284521
## untreated3fb untreated  paired-end               2         10542625 (x2)
## untreated4fb untreated  paired-end               2         12214974 (x2)
##              exon.counts
## treated1fb      15679615
## treated2fb      15620018
## treated3fb      12733865
## untreated1fb    14924838
## untreated2fb    20764558
## untreated3fb    10283129
```

```
## untreated4fb    11653031
```

As you can see, we have 7 samples in total, 3 of them are knockdown replicates (treated with siRNA) and 4 of them are untreated controls. Of note, some samples come from single-end sequencing, while others come from paired-end sequencing. This is an important detail to keep in mind when performing downstream analyses, as it may represent a source of technical variability that needs to be accounted for.

The count matrix has 14599 (genes) and 7 columns (samples).

If you notice, the annotation table row names contain the sample names. However, the column names of the count matrix do not match the sample names in the annotation table. Moreover, the order of the samples in the count matrix is different from the order of the samples in the annotation table.

This has to be fixed since DESeq2 requires that the column names of the count matrix match the row names of the annotation table, and that they are in the same order.

```r
#fix the sample names in the annotation table
rownames(pasAnno) <- gsub("fb", "", rownames(pasAnno))
#check if now the sample names match
all(rownames(pasAnno) %in% colnames(pasillaRawCounts))
```

```
## [1] TRUE
```

```r
#check if the order of the samples in the annotation table matches the order of the samples in the coun
all(rownames(pasAnno) == colnames(pasillaRawCounts))
```

```
## [1] FALSE
```

```r
#Fix the order
pasAnno <- pasAnno[colnames(pasillaRawCounts),]
#check again
all(rownames(pasAnno) == colnames(pasillaRawCounts))
```

```
## [1] TRUE
```

2. Filtering the genes

While DESeq2 performs independent filtering internally during the differential expression analysis, we apply a manual pre-filtering step to improve computational efficiency. By removing genes with consistently low counts, we significantly reduce the memory required for the Variance Stabilizing Transformation and ensure that the resulting visualizations are not obscured by background noise.

We will keep genes that have a count of at least 5 in at least 3 samples (since 3 is our smallest group size).

```r
# Step 1: Identify genes where at least 3 samples have a count of 5 or more
# We use the raw data frame 'pasillaRawCounts'
keep <- rowSums(pasillaRawCounts >= 5) >= 3
pasillaFilteredCounts <- pasillaRawCounts[keep, ]

cat("Genes before filtering:", nrow(pasillaRawCounts), "\n")
```

```
## Genes before filtering: 14599
```

```r
cat("Genes after filtering:", nrow(pasillaFilteredCounts), "\n")
```

```
## Genes after filtering: 8813
```

3. Normalization for library Size and RNA composition

The primary challenge in comparing RNA-seq samples is that they are rarely sequenced to the same depth.

Let's check the total number of reads in each sample by summing the counts across all genes for each sample.

```
colSums(pasillaFilteredCounts)
```

```
## untreated1 untreated2 untreated3 untreated4   treated1   treated2   treated3
##   13968882   21904354    8355889    9838733   18662789    9569133   10340920
```

For instance, sample untreated1 has 13968882, while untreated2 has 21904354 total reads. This means that untreated2 has a bigger library size and if we were to compare the raw counts directly, every gene in untreated2 would appear artificially upregulated compared to untreated1, simply because it has more reads overall.

Furthermore, because a sequencer has a finite sequencing depth, the genes effectively compete for available reads. If a few genes are extremely abundant in one condition, they will occupy a larger fraction of the total reads, reducing the counts available for the rest of the transcriptome.

To correct this, DESeq2 uses the Median-of-Ratios method:

1. The geometric mean of counts for every gene across all samples is calculated.
2. Each gene count in a sample is divided by the corresponding gene's geometric mean, resulting in a ratio for each gene in that sample.
3. The median value of all ratios for each sample is the size factor for that sample.

Other normalization exist:

- TMM (Trimmed Mean of M-values): the default for the edgeR package; it uses a weighted trimmed mean of the log expression ratios between samples. It is used for differential expression.
- CPM (Counts Per Million): the counts are scaled by the total number of reads. It is not used for differential expression.
- TPM (Transcripts Per Million): normalizes for gene length and library size. It is not used for differential expression.

Let's apply the DESeq2 normalization to our filtered count matrix.

```
# 1. set the condition as a factor in the annotation table
pasAnno$condition <- factor(pasAnno$condition, levels = c("untreated", "treated"))


# 2. Create the DESeqDataSet and define the design (by providing the column in the annotation table tha
dds <- DESeqDataSetFromMatrix(countData = pasillaFilteredCounts,
                              colData = pasAnno,
                              design = ~ condition)

# 3. Estimate Size Factors to normalize for depth and composition
dds <- estimateSizeFactors(dds)


# 4. Inspect the results
# A value > 1 means the sample was sequenced deeper than average; < 1 means shallower.
print(sizeFactors(dds))
```

```
## untreated1 untreated2 untreated3 untreated4   treated1   treated2   treated3
##  1.1409695  1.7946046  0.6477490  0.7511753  1.6345445  0.7610815  0.8321646
```

Good, now the size factors are stored in the `dds` object. However, the counts in the `dds` object are still the raw counts. To get the normalized counts, we need to use the `counts()` function.

```r
# Get the normalized counts using the size factors
pasillaNormCounts <- counts(dds, normalized = TRUE)
head(pasillaNormCounts, 4)
```

```
##              untreated1  untreated2  untreated3  untreated4    treated1
## FBgn0000008   80.633179   89.713353  117.329391   93.187304   85.650770
## FBgn0000017 4087.751593 4855.665585 5502.130931 4193.428671 3796.164506
## FBgn0000018  510.968949  424.048831  378.232906  412.686631  441.713259
## FBgn0000024    8.764476    6.129484    4.631423    3.993742    6.117912
##                treated2    treated3
## FBgn0000008  115.624938   84.117976
## FBgn0000017 4036.361483 4006.419010
## FBgn0000018  392.862006  370.119093
## FBgn0000024    9.197438    6.008427
```

4. Variance Stabilizing Transformation (VST)

Once normalized, we apply a Variance Stabilizing Transformation (VST) to ensure the variance is independent of the mean.

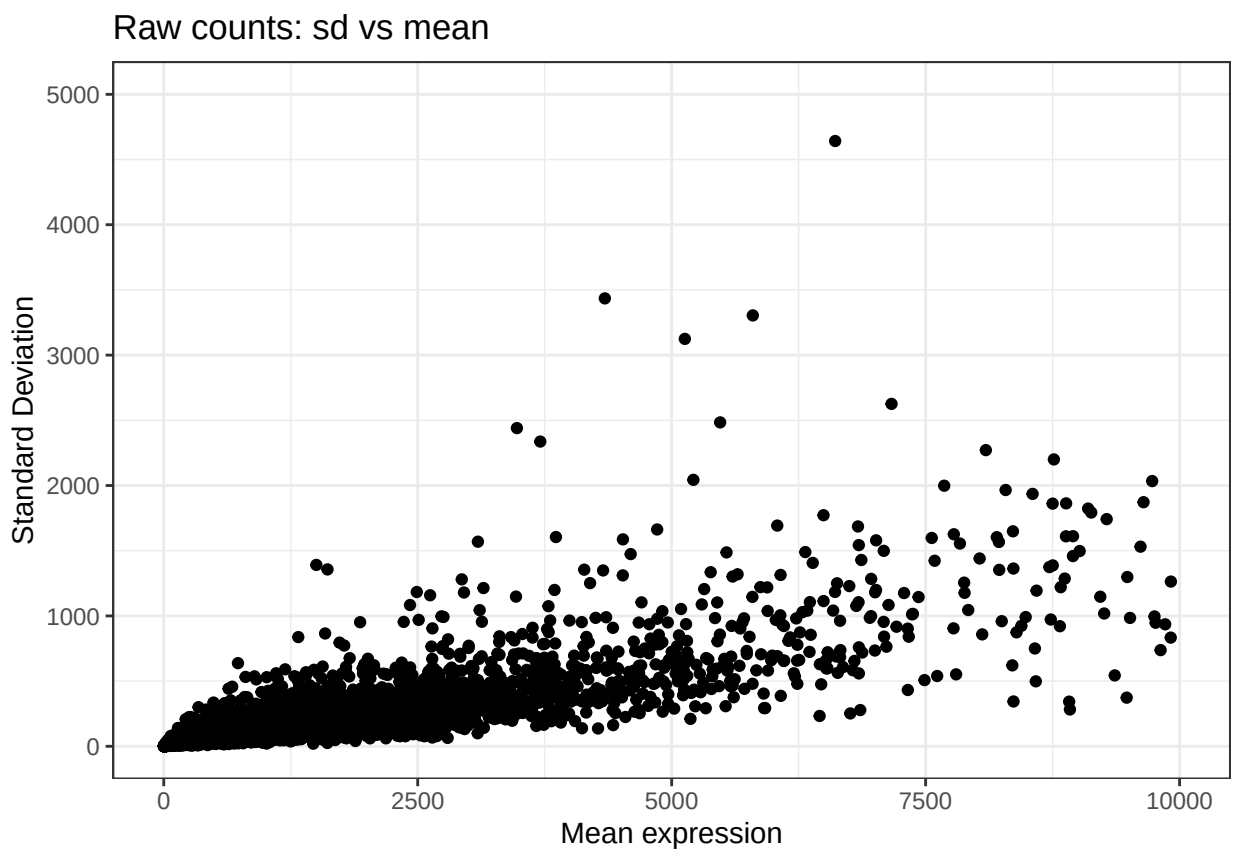
VST is a critical step because raw RNA-seq counts are fundamentally heteroscedastic. This means that the variance of a gene's count depends on its mean expression level: genes with high counts naturally have higher absolute variance (more "noise") than genes with low counts. Another approach to stabilize the variance is the regularized log transformation (rlog), which is more computationally intensive and is typically used for smaller datasets.

We can check the variance-mean relationship in the raw counts by plotting the variance against the mean for each gene.

```r
# for each gene, we retrieve the mean and the standard deviation (SD) across all samples, and we plot t
allGenes_mean <- rowMeans(pasillaNormCounts)
allGenes_sd <- rowSds(as.matrix(pasillaNormCounts))

df_normCounts_mean_sd <- data.frame(mean = allGenes_mean, sd = allGenes_sd)

ggplot(df_normCounts_mean_sd, aes(x = mean, y = sd)) +
  geom_point() +
  theme_bw() +
  xlim(c(0,10000)) +
  ylim(c(0,5000)) +
  xlab("Mean expression") +
  ylab("Standard Deviation") +
  ggtitle("Raw counts: sd vs mean")
```



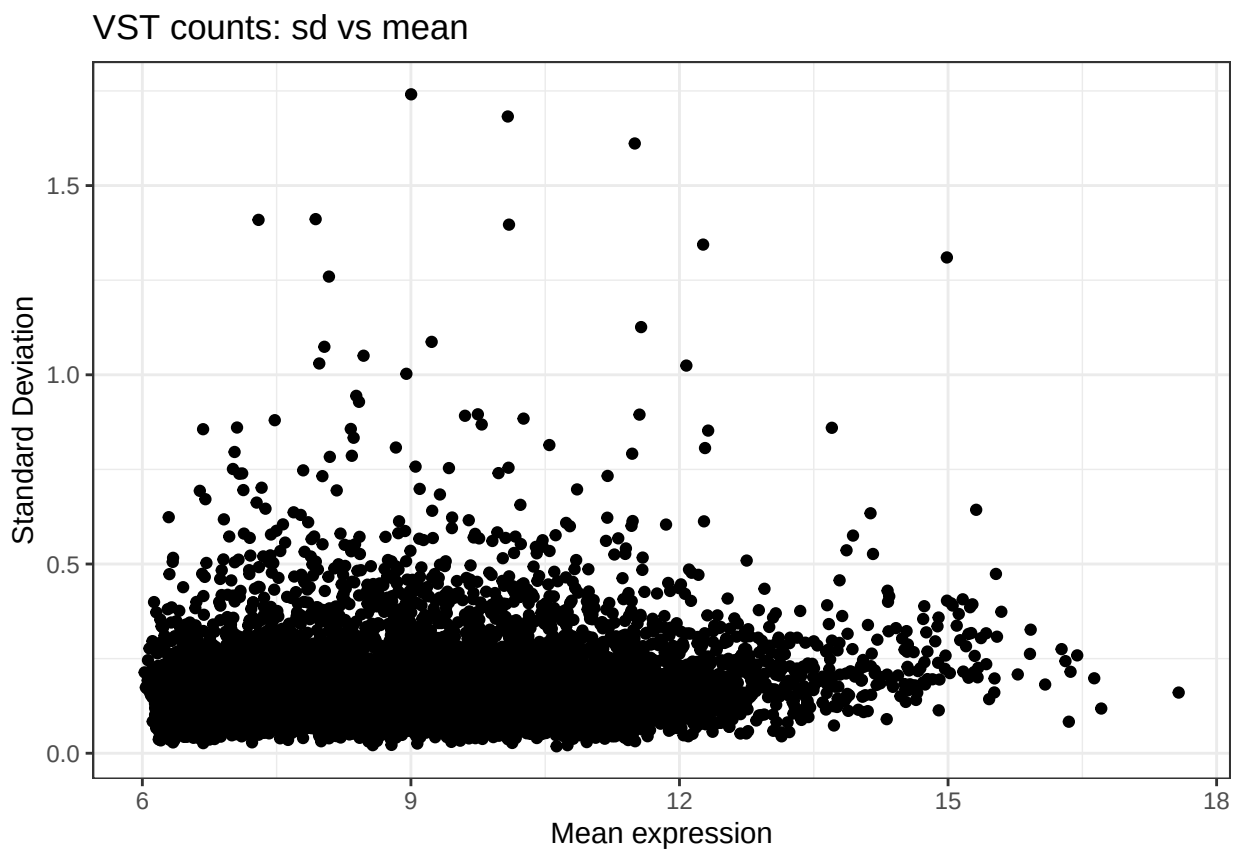
As you can see, for greater value of mean expression, the variance (or standard deviation) increases, which is a problem for downstream analyses that assume homoscedasticity (constant variance across the range of mean expression levels).

After applying the VST, we can check the variance-mean relationship again.

```
# Apply VST
vst_counts <- DESeq2::vst(dds, blind = T)

# VST counts mean expression Vs standard Deviation (SD)
vst_mean <- rowMeans(assay(vst_counts))
vst_sd <- rowSds(assay(vst_counts))
df_vst_mean_sd <- data.frame(mean = vst_mean, sd = vst_sd)

ggplot(df_vst_mean_sd, aes(x = mean, y = sd)) +
  geom_point() +
  theme_bw() +
  xlab("Mean expression") +
  ylab("Standard Deviation") +
  ggtitle("VST counts: sd vs mean")
```



As you can see, after applying the VST, the variance is stabilized across the range of mean expression levels, making it more suitable for downstream analyses like PCA and clustering.

5. Exploratory data analysis (EDA) on the normalized data

5.1 PCA

PCA is an unsupervised analysis, meaning it organizes our samples based on the inherent variance in the data without being told which group is which. By collapsing thousands of gene expression variables into two dimensions, it reveals the primary drivers of difference across the experiment. In a successful, well-controlled study, biological replicates should cluster tightly together, indicating that their profiles are consistent and reproducible.

The greatest distance between these clusters should ideally correspond to the treatment groups, such as the difference between pasilla knockdown and control samples. This makes PCA an essential diagnostic tool for identifying outliers that fail to group with their replicates or for spotting batch effects, where samples cluster by technical factors like sequencing date or library type rather than biology. If the biological signal is stronger than the technical noise, the PCA will clearly separate your experimental conditions along the first principal component.

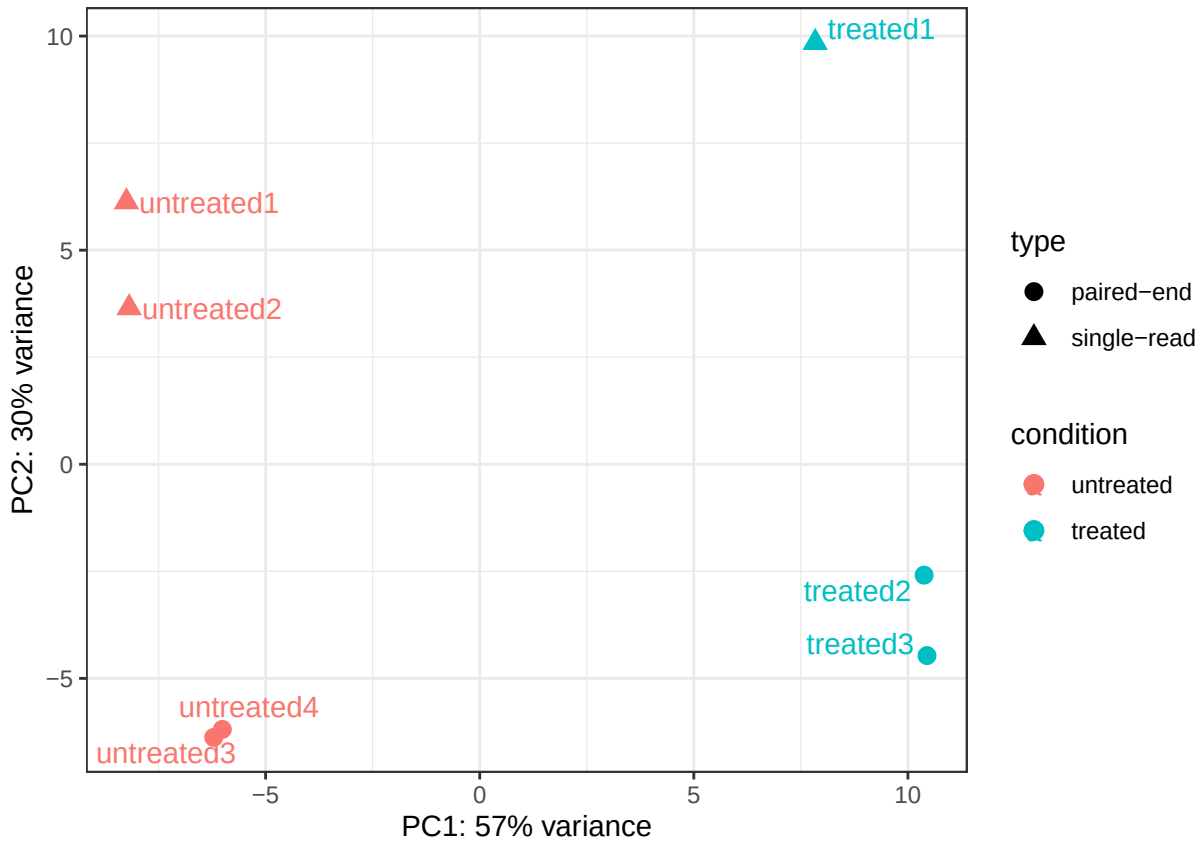
```
# Perform PCA on the VST-transformed data
pcaData <- plotPCA(vst_counts, intgroup=c("condition", "type"), returnData=TRUE)
```

```
## using ntop=500 top features by variance
```

```
percentVar <- round(100 * attr(pcaData, "percentVar"))

ggplot(pcaData, aes(PC1, PC2, color=condition, shape=type)) +
```

```
  geom_point(size=3) +
  theme_bw() +
  geom_text_repel(data = pcaData, aes(label = name)) +
  xlab(paste0("PC1: ",percentVar[1],"% variance")) +
  ylab(paste0("PC2: ",percentVar[2],"% variance")) +
  coord_fixed()
```



As you can see, the first principal component (PC1) captures the largest source of variance in the data, which in this case is the treatment condition (knockdown vs control). The second principal component (PC2) captures the next largest source of variance, which in this case is related to the library type (single-end vs paired-end). The samples cluster according to their treatment condition, indicating that the biological signal is stronger than the technical noise. However, there is also some separation based on library type, suggesting that this technical factor may also be contributing to the variance in the data. This has to be kept in mind when interpreting the results of downstream analyses, as it may represent a confounding factor that needs to be accounted for (e.g., by including library type as a covariate in the design formula for differential expression analysis).

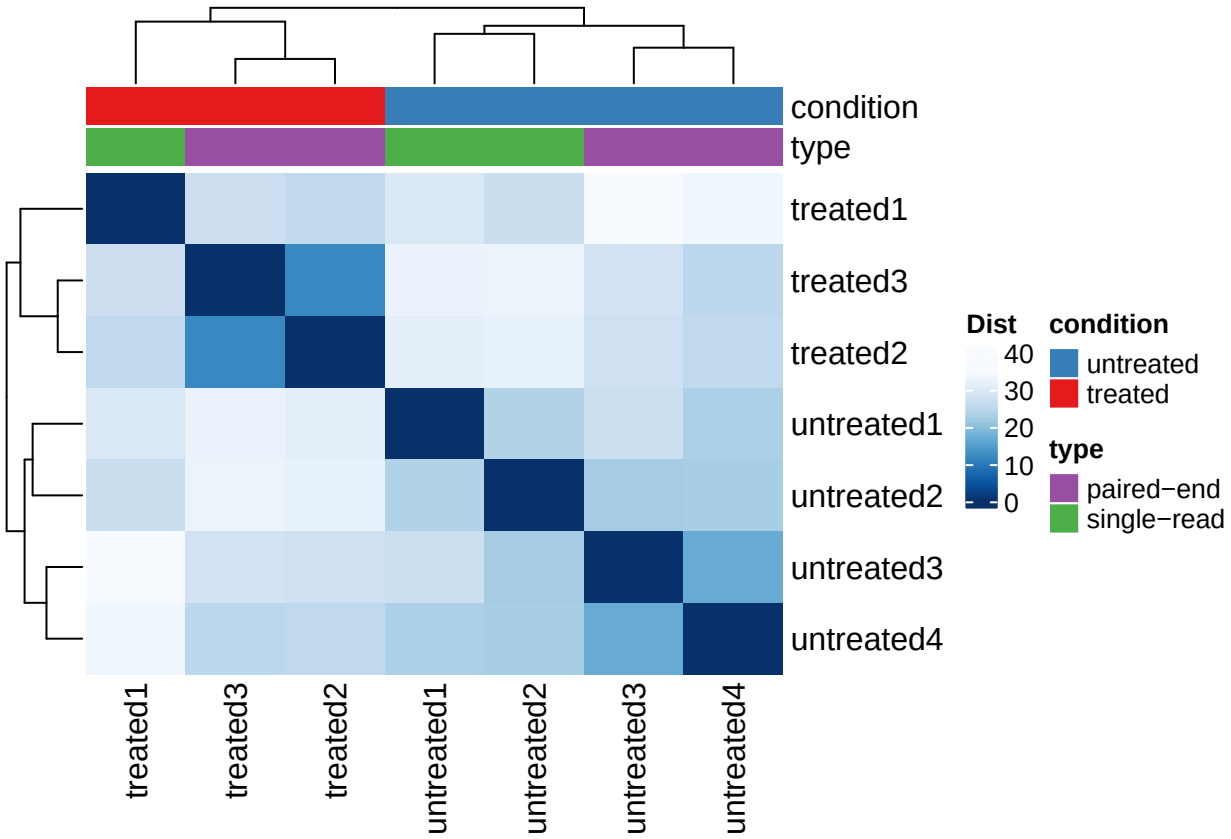
5.2 Sample distance heatmap

Another way to visualize the relationships between samples is to create a heatmap of the sample-to-sample distances. This can help identify clusters of similar samples and potential outliers. For this, we use the **dist** function to calculate the distance matrix between samples based on the VST-transformed counts, and then we visualize it using the **ComplexHeatmap** package.

```
# Calculate the distance matrix between samples
sample_dist <- as.matrix(dist(t(assay(vst_counts))))
```

```r
ha <- HeatmapAnnotation(df = as.data.frame(colData(dds)[, c("condition", "type")]),
                        col = list(condition = c("untreated" = "#377eb8", "treated" = "#e41a1c"),
                                   type = c("single-read" = "#4daf4a", "paired-end" = "#984ea3")))

Heatmap(sample_dist, name = "Dist", top_annotation = ha,
        col = colorRampPalette(rev(brewer.pal(9, "Blues")))(100))
```



Hierarchical clustering of the distance matrix should show that samples cluster according to their treatment condition, with the three knockdown replicates clustering together and the four control samples clustering together. However, we may also see some separation based on library type, which again suggests that this technical factor may be contributing to the variance in the data.

5.3 Heatmap of the most variable genes

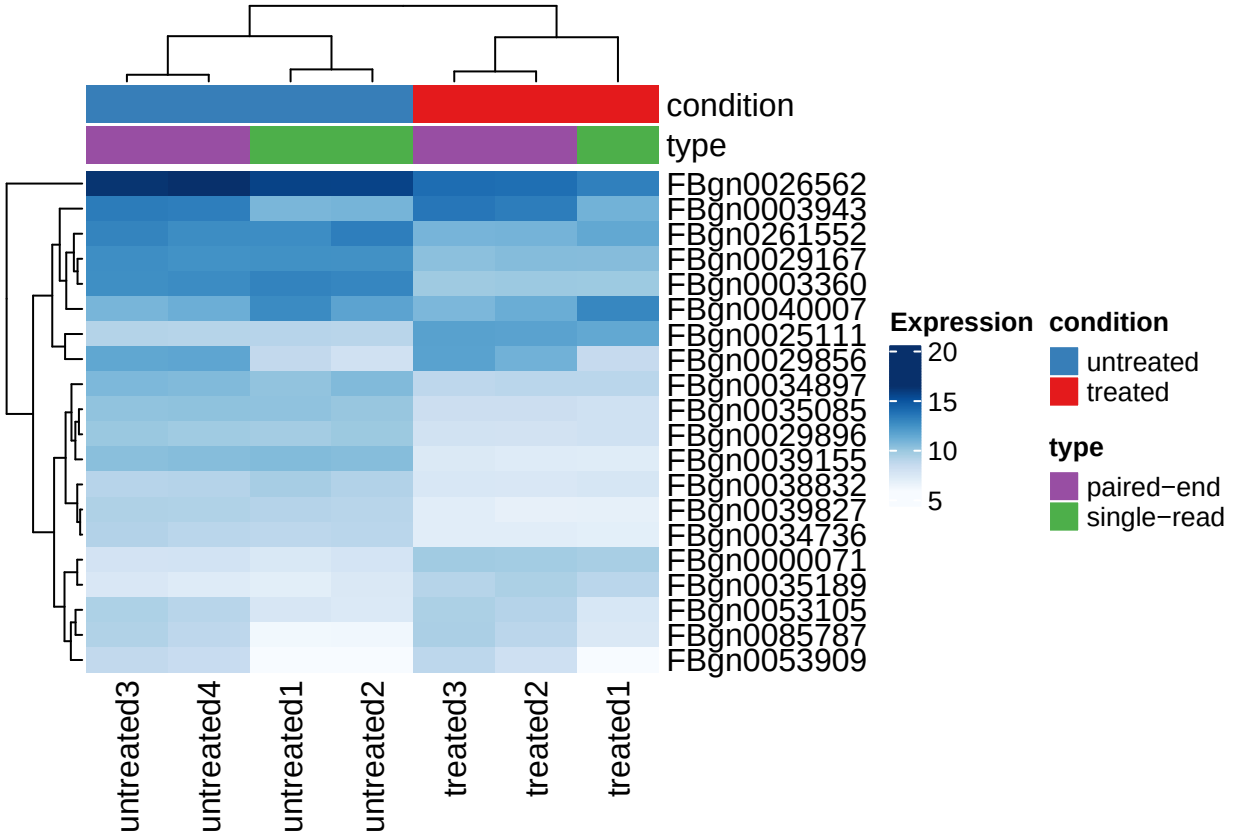
To further explore the data, we can create a heatmap of the most variable genes across the samples. This can help identify patterns of gene expression that differentiate the treatment groups.

```r
# Select the top 20 most variable genes
topVarGenes <- head(order(rowVars(assay(vst_counts)), decreasing = TRUE), 20)

# Create a heatmap of the top variable genes

ha_VarGenes <- HeatmapAnnotation(df = as.data.frame(colData(dds)[, c("condition", "type")]),
                        col = list(condition = c("untreated" = "#377eb8", "treated" = "#e41a1c"),
                                   type = c("single-read" = "#4daf4a", "paired-end" = "#984ea3")))
```

```r
Heatmap(assay(vst_counts)[topVarGenes, ], name = "Expression", top_annotation = ha_VarGenes,
        col = colorRampPalette(brewer.pal(9, "Blues"))(100))
```



We can see that the expression patterns of the top variable genes differentiate the treatment groups, with some genes being more expressed in the knockdown samples and others being less expressed. For instance, the fly gene sesB (FBgn0003360) appears to be less expressed in the knockdown samples compared to the controls, while Ant2 (FBgn0025111) becomes upregulated, in line with the paper (Figure 3A).

However, we may also see some patterns that are associated with library type (FBgn0029856), which again suggests that this technical factor is contributing to the variance in the data.

Note that to interpret the biological significance of these patterns, we would need to perform a differential expression analysis to identify which genes are significantly differentially expressed between the treatment groups. The analysis of the most variable genes is just an exploratory step to visualize the data and generate hypotheses for further investigation.